

PART 6 — Python Internals

This is the senior-interview layer.

At this point, don't memorize isolated definitions. Build one mental model:

Python code → CPython compiler → bytecode → interpreter → objects in memory → references → reference counting + cyclic GC → CPU execution.

One important modern update: **the classic GIL story is no longer the whole story**. CPython has supported optional free-threaded builds since Python 3.13, so in a senior interview in 2026 you should know both the traditional GIL model and the newer free-threaded model. ([Python documentation](#))

How CPython Works Under the Hood



Memory Management



Performance



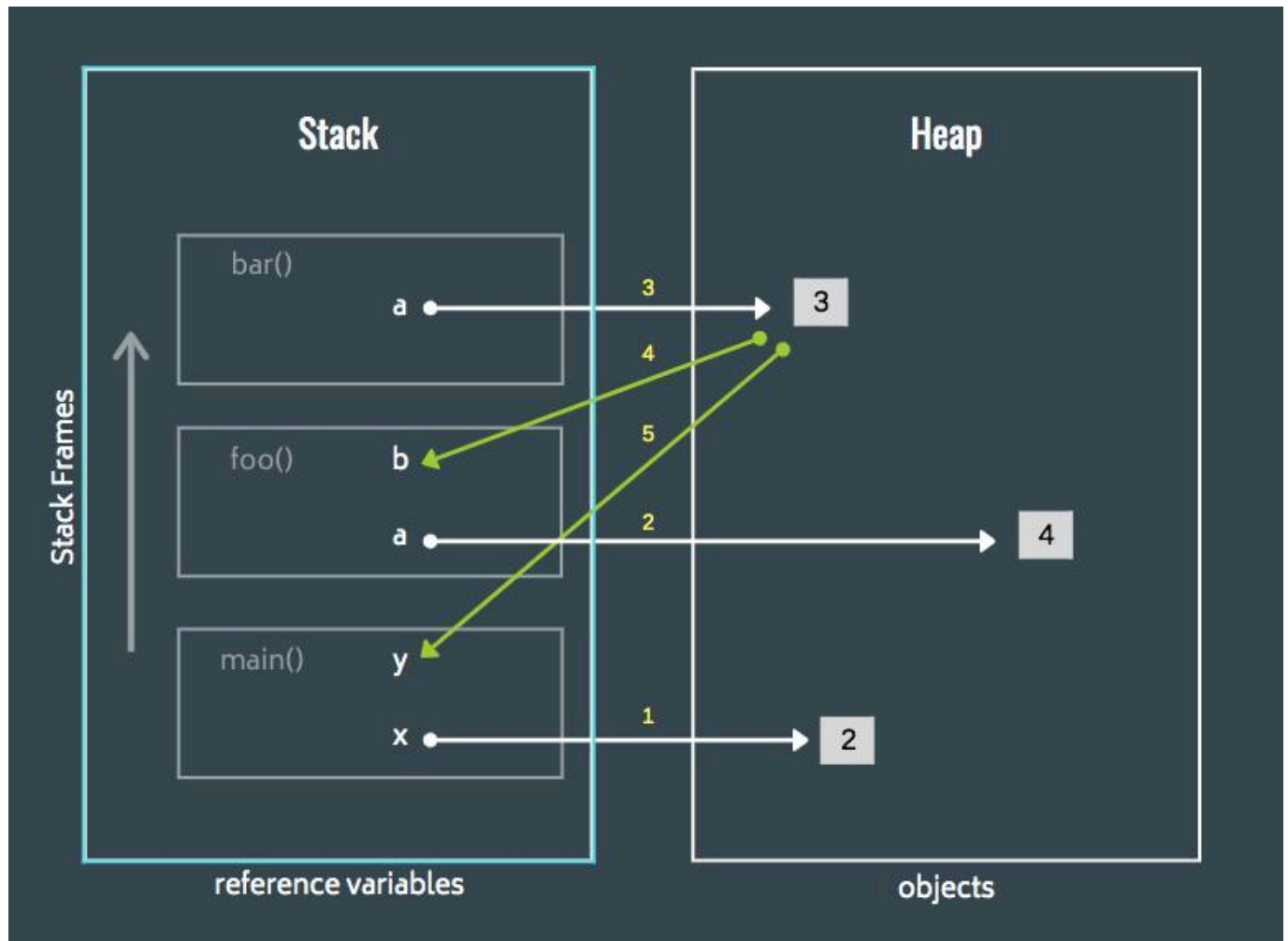
The GIL



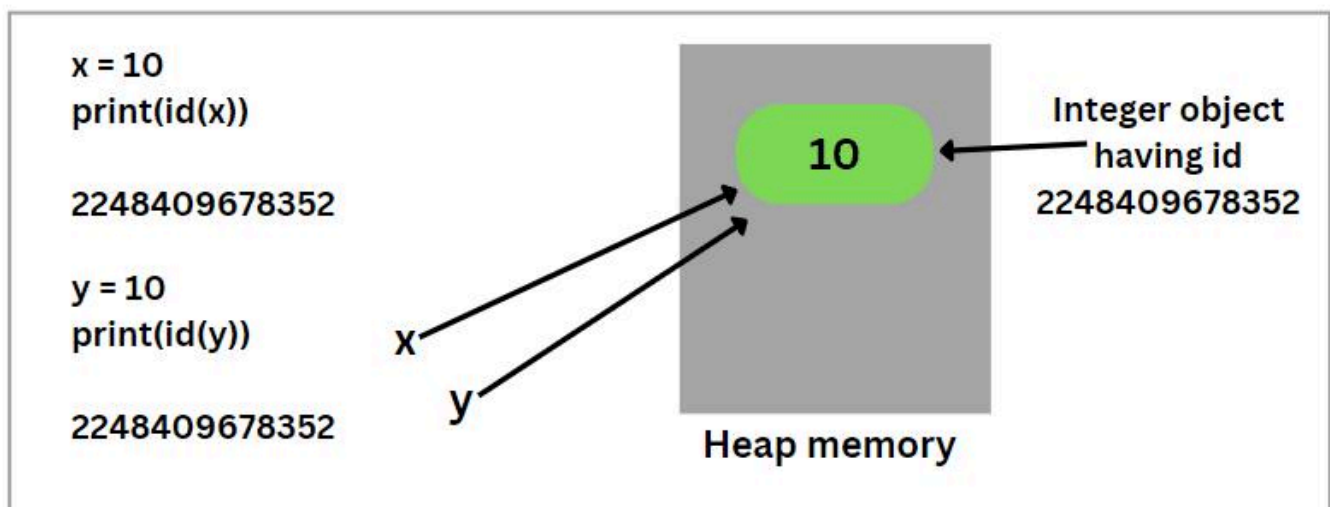
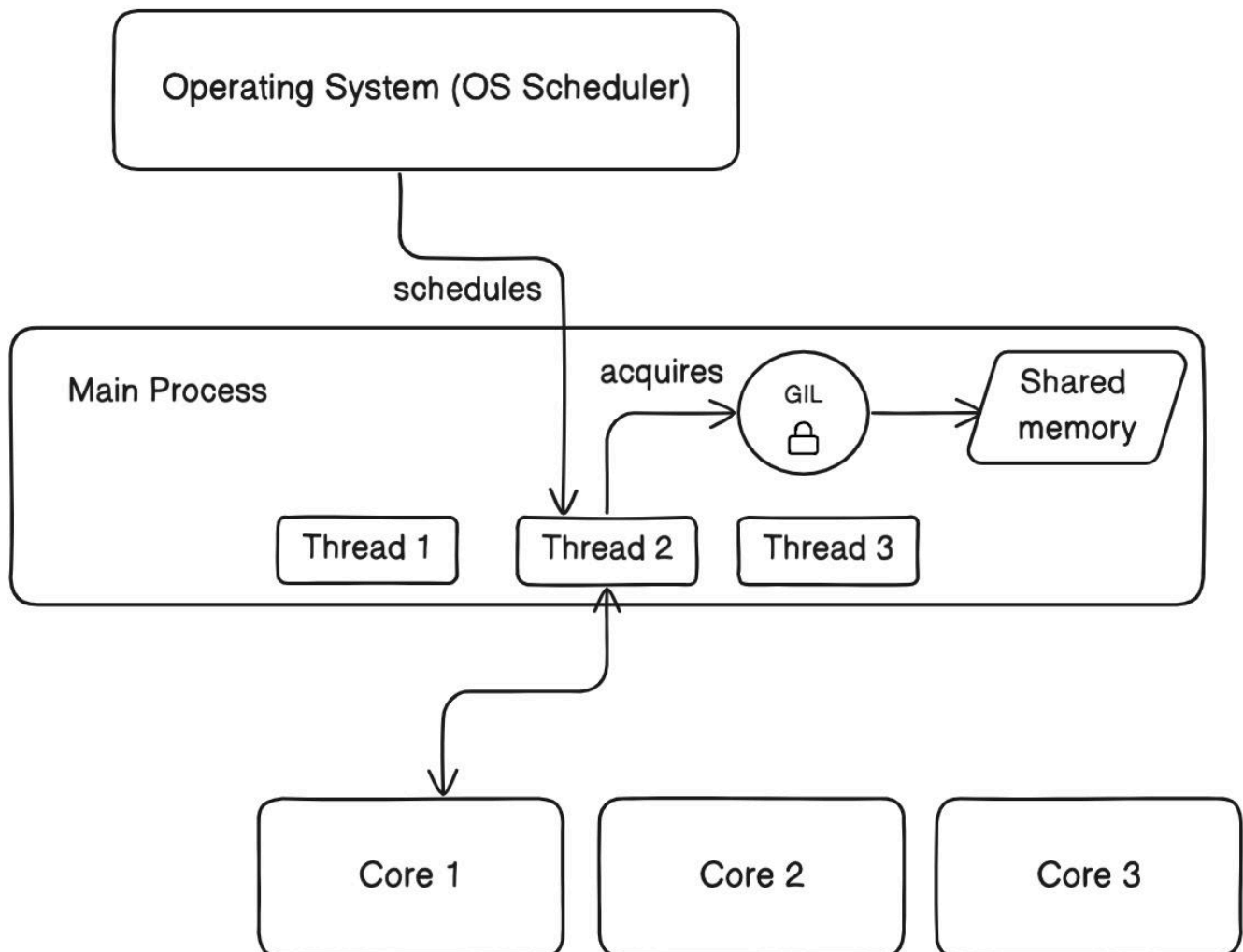
Memory & GC

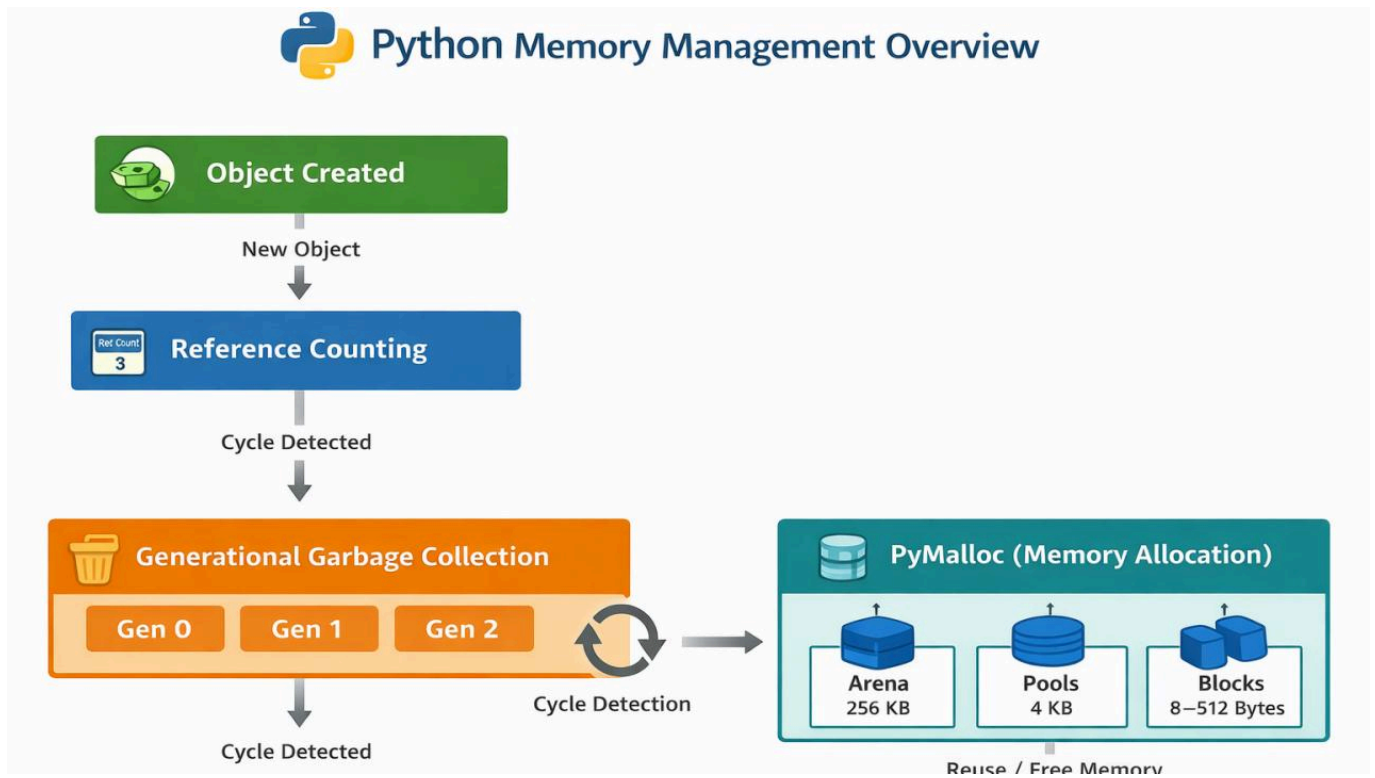


Scaling

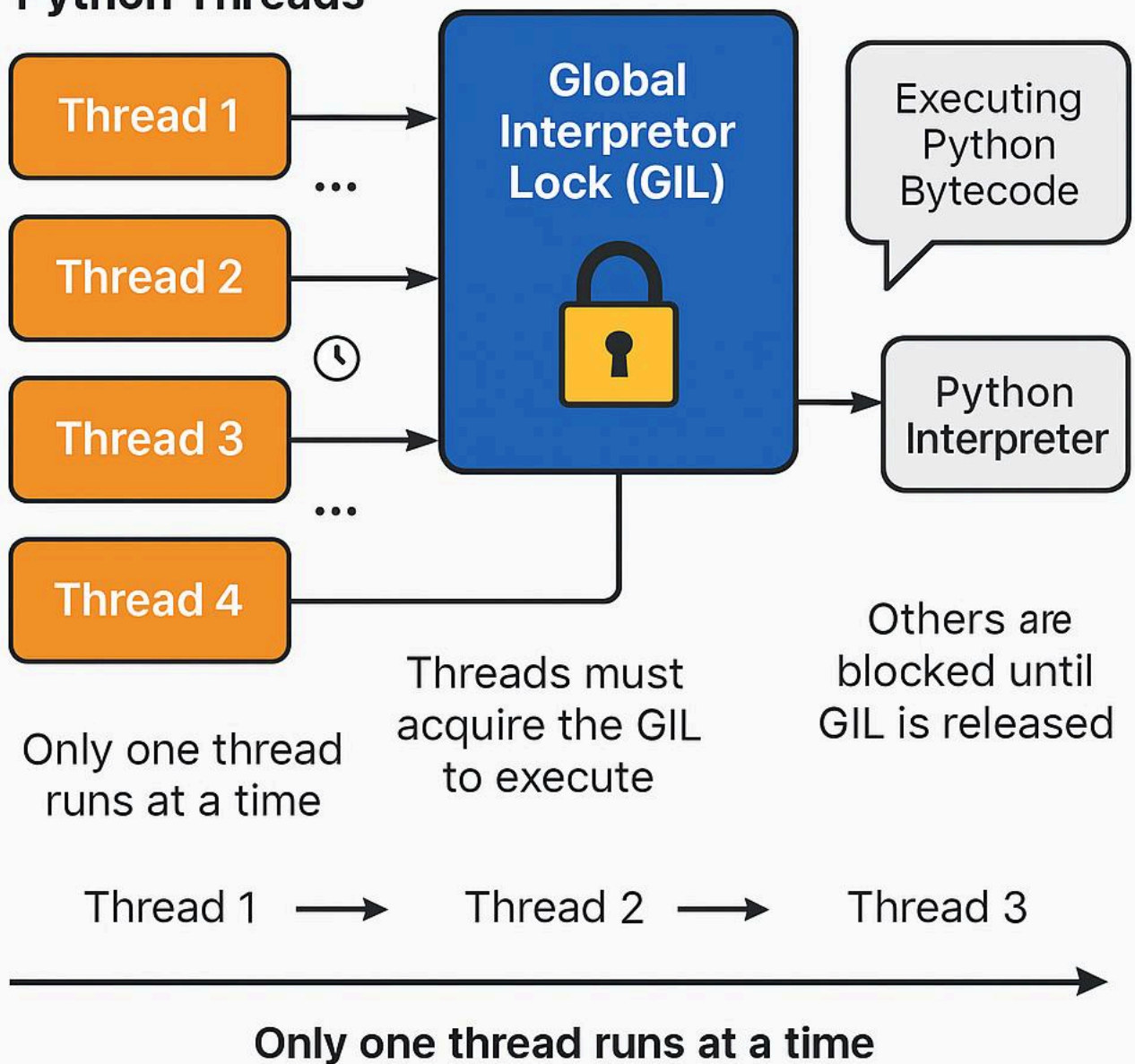


Multi threading (Python - GIL)



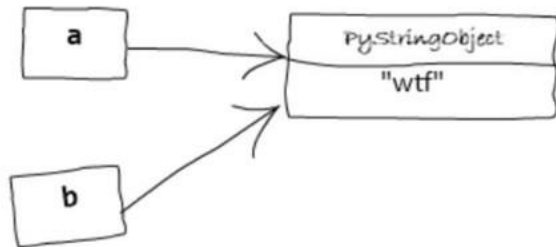


Python Threads



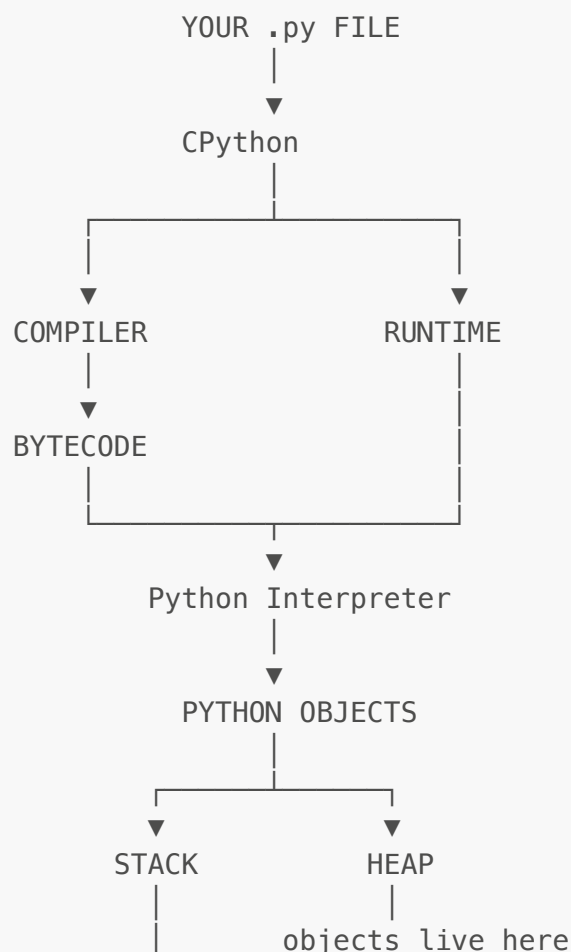
Implicit String interning

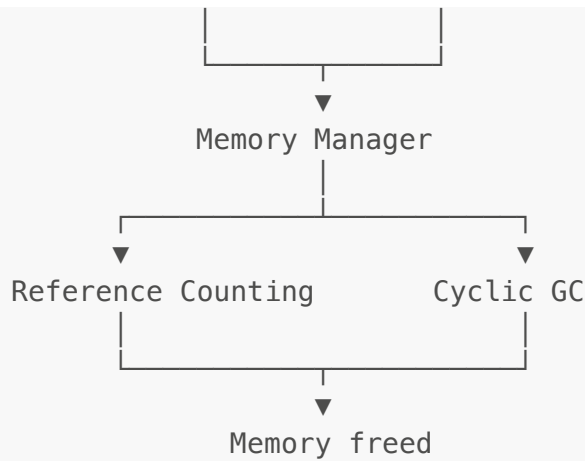
- Such behavior is due to CPython optimization (called string interning) that tries to use existing immutable objects in some cases rather than creating a new object every time.
- After being interned, many variables may point to the same string object in memory (thereby saving memory).



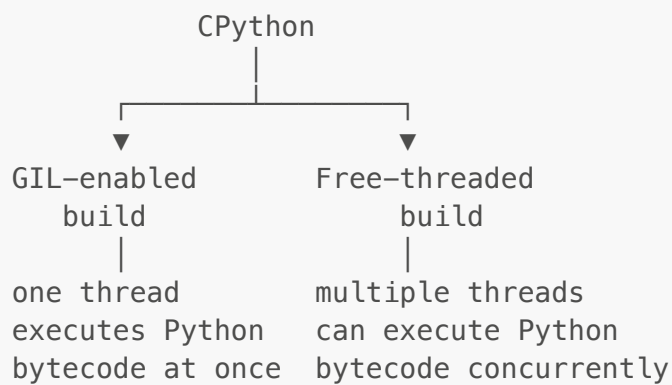
THE MASTER PICTURE

Before going topic by topic, memorize this:





And threads:



1. Python Memory Management

Python manages memory for you.

You don't normally write:

```
malloc(...)  
free(...)
```

Instead:

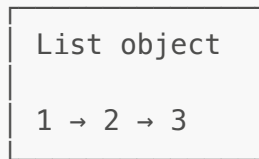
```
x = [1, 2, 3]
```

Python creates and manages the object.

Conceptually:

```
x = [1,2,3]
```

x
|
▼



HEAP

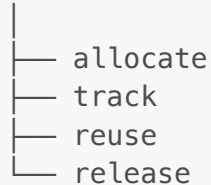
CPython maintains a **private heap containing Python objects and data structures**, managed by the Python memory manager. CPython also has specialized allocators; for example, the traditional GIL-enabled build uses `pymalloc` by default for relevant allocation domains, while free-threaded builds use `mimalloc` by default. ([Python documentation](#))

Mental model

Python program



Python Memory Manager



Senior interview point

Don't say:

"Python puts everything on the heap."

That's an oversimplification.

Better:

Python objects are managed in the interpreter's private heap, while execution also uses stack-like execution state.

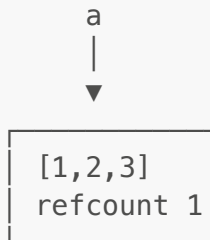
2. Reference Counting

This is one of the most important CPython internals.

Imagine:

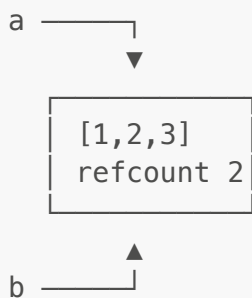
```
a = [1, 2, 3]
```

Conceptually:



Now:

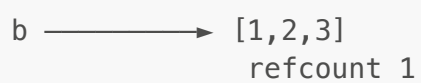
```
b = a
```



Now:

```
del a
```

Conceptually:



Then:

```
del b
```

No references remain:

```
[1,2,3]
refcount 0
  |
  ▼
DEALLOCATE
```

CPython's C API exposes operations such as `Py_INCREF()` and `Py_DECREF()` for managing strong references. When the last strong reference is released and the reference count reaches zero, the object's deallocation function is invoked. ([Python documentation](#))

🔥 Mental model

```
NEW REFERENCE
  ↓
refcount + 1

REFERENCE RELEASED
  ↓
refcount - 1

refcount == 0
  ↓
object can be deallocated
```

Important modern caveat

In current CPython, don't assume every observed reference count is a simple literal count. Some objects can be **immortal**, and `sys.getrefcount()` has caveats. ([Python documentation](#))

3. Garbage Collection

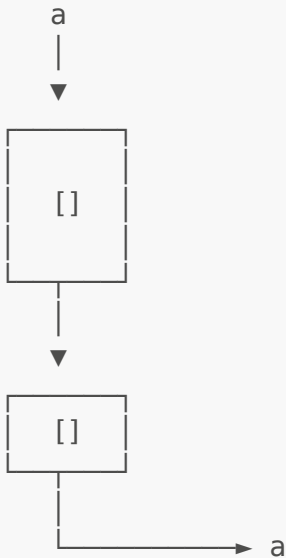
Reference counting has a problem.

Circular references.

```
a = []
b = []
```

```
a.append(b)
b.append(a)
```

Visual:



Even if your program no longer has external references to these objects, they can reference each other.

So:

```
reference counting
      +
cyclic garbage collector
      ↓
better memory reclamation
```

CPython's cyclic garbage collector **supplements reference counting** and is specifically useful for unreachable reference cycles. ([Python documentation](#))

You can interact with it:

```
import gc

gc.collect()
```

You can inspect:

```
gc.isenabled()
```

and:

```
gc.disable()
```



Senior answer

If interviewer asks:

```
"Does Python use garbage collection?"
```

Don't simply say:

```
"Yes."
```

Say:

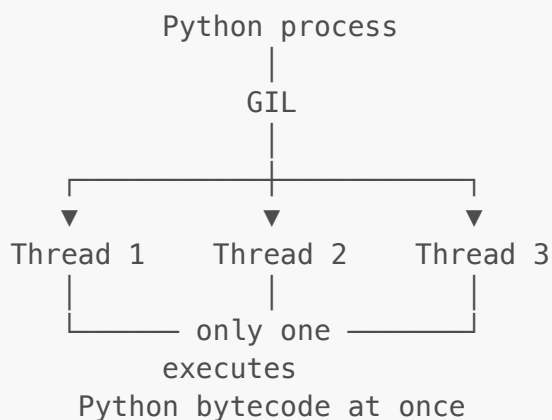
CPython primarily uses reference counting for immediate reclamation, supplemented by a cyclic garbage collector to detect unreachable reference cycles.

That's a much stronger answer.

4. GIL

GIL = **Global Interpreter Lock**.

Traditional CPython:



So:

```
Thread 1 → Python code
Thread 2 → waiting
Thread 3 → waiting
```

Then the lock can move between threads.

The GIL historically simplifies CPython's object model and means that, in a normal GIL-enabled build, only one thread executes Python bytecode at a time. It can be released around blocking I/O and by some native extensions doing intensive work. ([Python documentation](#))

! But this changed

Starting with **Python 3.13**, CPython supports a **free-threaded build** where the GIL can be disabled. Multiple threads can then execute Python code concurrently on different CPU cores. ([Python documentation](#))

So don't give this outdated interview answer:

"Python cannot run multiple threads in parallel."

Better:

Traditional GIL-enabled CPython allows only one thread to execute Python bytecode at a time, but modern CPython also supports optional free-threaded builds where the GIL is disabled and Python threads can execute concurrently.

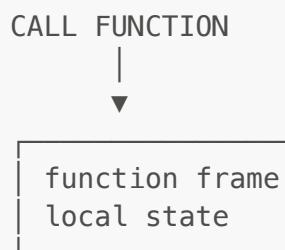
This distinction is **very senior-level**.

5. Stack vs Heap

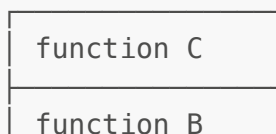
This concept is often oversimplified.

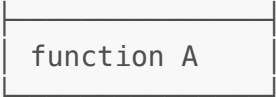
Stack

Think:



Nested calls:





A diagram showing a rectangular box with a double border. Inside the box, the text "function A" is written.

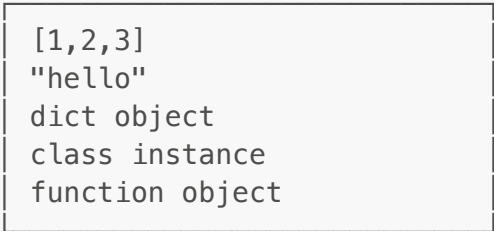
```
function A
```

When functions return, their execution state is unwound.

Heap

Objects are managed in Python's private heap:

HEAP



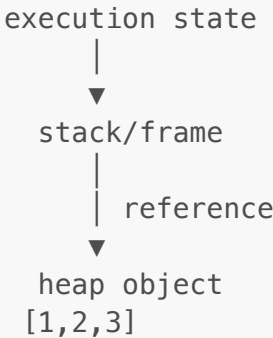
A diagram showing a rectangular box with a double border. Inside the box, the following text is listed vertically: "[1,2,3]", "hello", "dict object", "class instance", and "function object".

```
[1,2,3]  
"hello"  
dict object  
class instance  
function object
```

Example:

```
def test():  
    x = [1, 2, 3]
```

Mental model:



A diagram illustrating the mental model of Python's execution architecture. It shows a vertical flow: "execution state" at the top, followed by a downward arrow, then "stack/frame", followed by another downward arrow, then "reference", followed by a third downward arrow, then "heap object", and finally "[1,2,3]" at the bottom.

```
execution state  
  |  
  ▼  
stack/frame  
  |  
  reference  
  ▼  
heap object  
[1,2,3]
```

⚠ Senior-level nuance

Avoid saying:

"All local variables are on the stack and all objects are on the heap."

Python's actual execution architecture is more nuanced than that simplified C/C++ model. CPython uses interpreter frames and object references, while object memory is managed through Python's memory-management system.

6. Object Interning

This is where `is` can surprise you.

Python may **reuse certain immutable objects**.

Example:

```
a = 10
b = 10

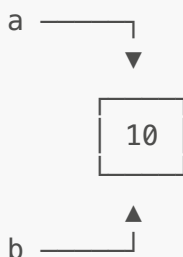
print(a is b)
```

You may get:

```
True
```

because CPython can reuse an existing integer object.

Visual:



Instead of:

```
a → 10
b → 10
```

two separate objects.

Strings can also be interned in certain circumstances.



Critical interview point

Never write code relying on:

```
a is b
```

to test value equality.

Use:

```
a == b
```

`is` means:

```
same object?
```

`==` means:

```
equal value?
```

Interning is an **implementation optimization**, not something you should rely on for general identity behavior.

7. Python Bytecode

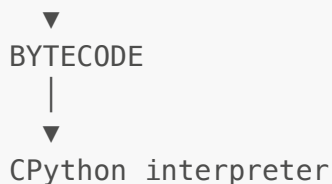
Python source:

```
def add(a, b):  
    return a + b
```

is compiled into bytecode instructions that CPython's interpreter executes.

Visual:

```
SOURCE  
  |  
  ▼  
def add(a,b):  
    return a+b  
  |  
  ▼  
COMPILER  
  |
```



You can inspect it:

```
import dis

def add(a, b):
    return a + b

dis.dis(add)
```

You'll see instructions such as:

```
LOAD_FAST
BINARY_OP
RETURN_VALUE
```

The exact instructions vary between Python versions.

The official documentation explicitly describes bytecode as a **CPython implementation detail** and warns that it can change between Python releases and isn't guaranteed to be portable across Python VMs. ([Python documentation](#))



Important

Don't memorize:

```
LOAD_FAST
BINARY_OP
...
```

Instead understand:

```
Python source
  ↓
compiler
  ↓
bytecode
  ↓
interpreter
```

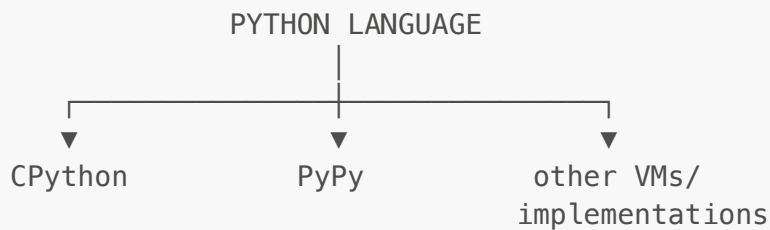
8. CPython

This distinction is extremely important.

Python is a language.

CPython is one implementation of that language.

Think:



CPython is the implementation most people mean when discussing Python's traditional reference-counting memory model, GIL, bytecode interpreter, and C API.

Why "CPython" matters

When someone says:

"Python has a GIL."

A senior engineer should mentally ask:

Which Python implementation/build?

Because the GIL is specifically a CPython runtime concept, and current CPython also has free-threaded builds. ([Python documentation](#))

9. `__name__ == "__main__"`

This looks simple but is very important for understanding modules.

Suppose:

```
# app.py  
print(__name__)
```

If you execute:

```
python app.py
```

Python sets:

```
__name__ = "__main__"
```

So:

```
python app.py
  |
  ▼
__name__ = "__main__"
```

But if another file does:

```
import app
```

then:

```
import app
  |
  ▼
__name__ = "app"
```

Therefore:

```
if __name__ == "__main__":
    main()
```

means:

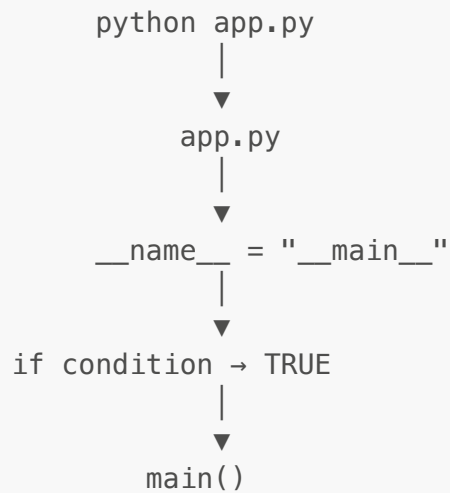
Run this block only when this file is executed directly, not when it is imported.

The official Python tutorial confirms that running a module as a script sets its `__name__` to `"__main__"`, while importing it gives it its module name. ([Python documentation](#))

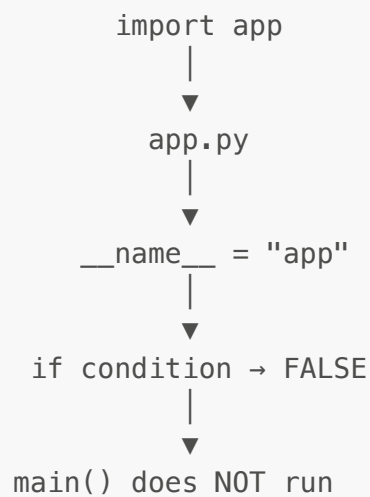


Visualizing it

Direct execution

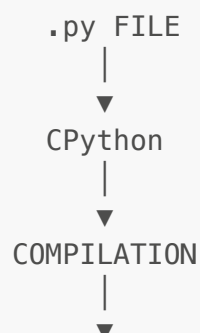


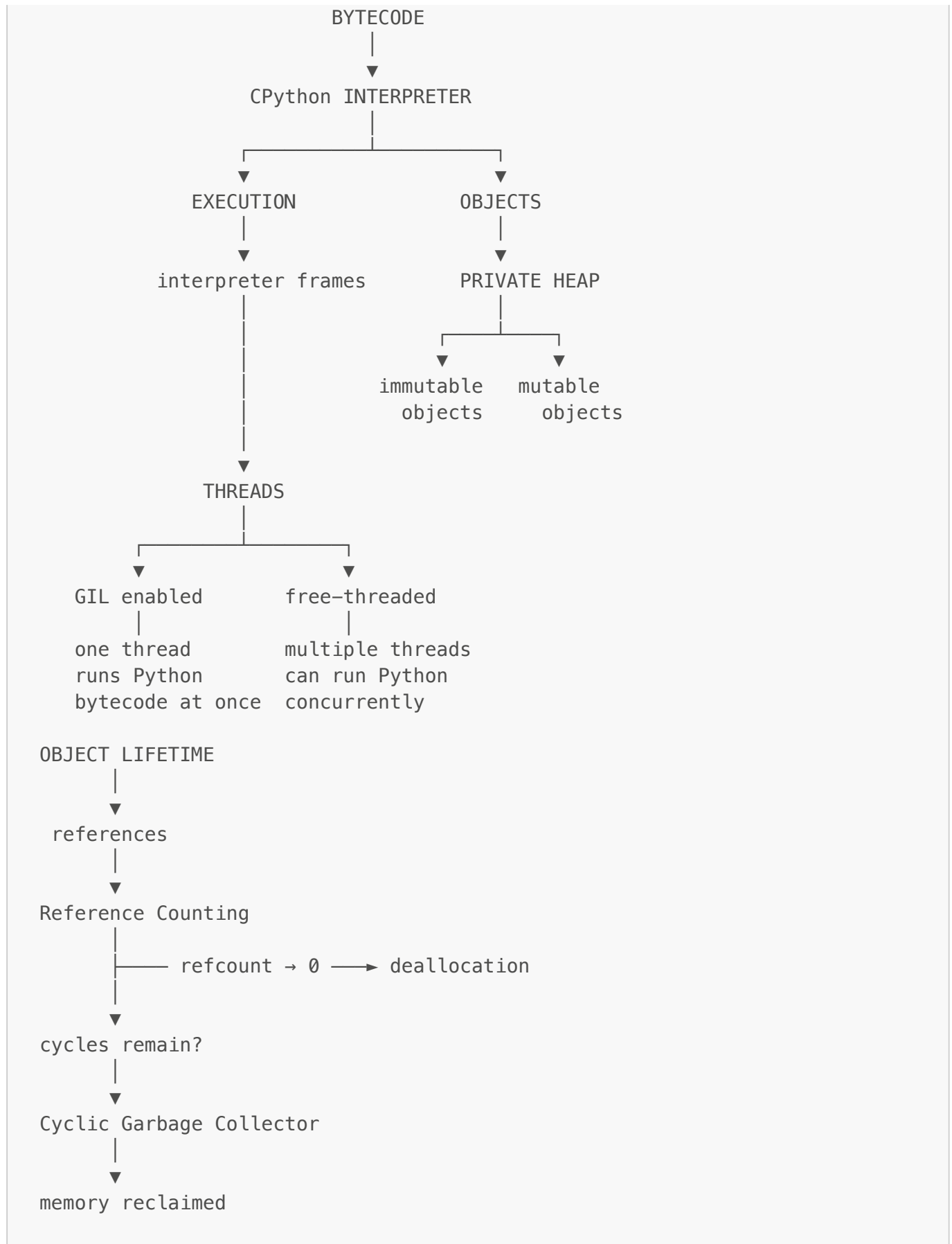
Import



THE COMPLETE INTERNALS MAP

Now put everything together:





Topic	Strong mental model
Memory Management	Python runtime manages a private heap of objects
Reference Counting	Track strong references to objects
Garbage Collection	Supplements refcounting, especially for cycles
GIL	Traditional CPython lock limiting Python bytecode execution to one thread at a time
Free-threaded CPython	Optional modern CPython build where GIL can be disabled
Stack	Execution/frame state
Heap	Python objects and managed memory
Interning	Reuse of certain immutable objects
Bytecode	Intermediate CPython instructions
CPython	A Python implementation
<code>__name__</code>	Module's execution/import identity
<code>__main__</code>	Indicates the module is being run as the main script



7 QUESTIONS YOU SHOULD BE ABLE TO ANSWER

If you're preparing for a **senior Python interview**, these are the questions I'd drill:

1. Why does Python need garbage collection if it has reference counting?

```

Reference counting
    ↓
great for immediate cleanup
    ↓
BUT
    ↓
cannot by itself resolve cycles
    ↓
cyclic GC

```

2. What exactly does the GIL protect?

Not simply "your variables."

The traditional GIL protects CPython's runtime while a thread is executing Python/C API operations that require it, helping make the object model manageable. ([Python documentation](#))

3. Does the GIL mean Python threads are useless?

No.

I/O-bound workloads can benefit from threads because the GIL is released around blocking I/O. Native extensions may also release it. And modern free-threaded CPython can run Python code concurrently across threads. ([Python documentation](#))

4. Why can this happen?

```
a = 10
b = 10

a is b
```

Potentially:

```
True
```

because of object reuse/interning.

But:

```
a == b
```

is the correct value comparison.

5. Is bytecode the same as machine code?

```
Python source
  ↓
Python bytecode
  ↓
CPython interpreter/runtime
  ↓
native machine execution
```

Bytecode is not the same thing as CPU machine code.

6. What is CPython?

CPython is the primary implementation of Python, implemented largely in C, with its own compiler, interpreter/runtime, object model, memory manager, and C API.

7. Explain this line:

```
if __name__ == "__main__":
```

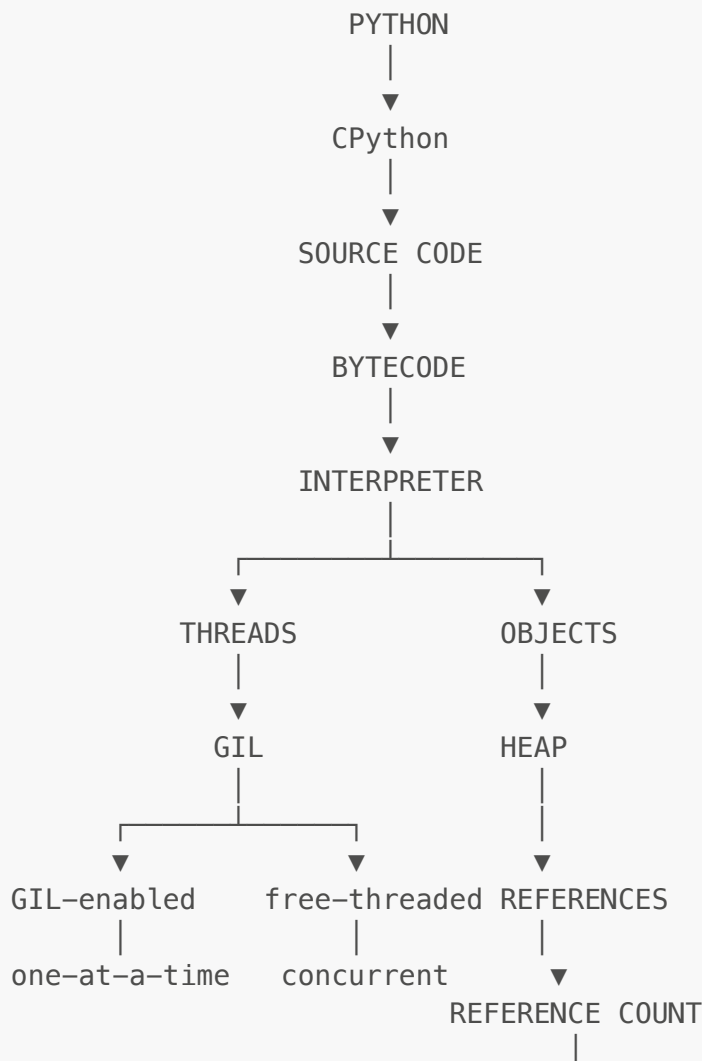
Best senior-level answer:

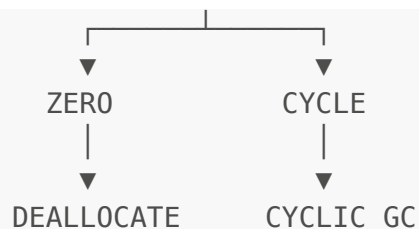
Every module has a `__name__`. When the file is executed directly, its `__name__` is set to `"__main__"`. When imported, it normally gets its module name. Therefore the condition lets us separate reusable module code from direct-script entry-point code. ([Python documentation](#))



FINAL MEMORY PALACE

Memorize this picture:





And one final connection to your previous parts:

```
PART 1
Objects + references
    ↓
PART 3
Functions + scope
    ↓
PART 5
Objects + classes
    ↓
PART 6
How CPython actually manages those objects
    ↓
PART 7
Exceptions / files / modules / imports
    ↓
PART 8
Iterators / generators / decorators / context managers
```

If you truly understand Part 6, you stop thinking of Python as "magic." You start seeing it as a runtime managing objects, references, frames, bytecode, threads, and memory. That is exactly the mental shift that helps in senior-level Python interviews.